AutoStack

Autonomous Vehicle Simulation & Validation Platform

Full Product Design Document

Version 1.0 (MVP) — Version 2.0 (Enterprise SaaS)

Product Architect:	Somasekhar Eruvuri, Senior Platform/SRE Engineer
Engineering Team:	3 CS Interns (3rd Year)
Industry:	Autonomous Vehicles, ADAS, Robotics
Target Buyers:	Automotive OEMs, ADAS Startups, Research Labs
v1 Duration:	2 Months (Prototype)
v2 Duration:	12 Months (Enterprise Platform)
Document Date:	May 2026

Executive Summary

AutoStack is a cloud-native AV simulation and ML validation platform that enables automotive and robotics teams to design, run, and validate autonomous driving scenarios at scale — without dedicated simulation hardware. It combines a 3D physics simulation engine, sensor fusion, ML-based perception, scenario management, and a CI/CD validation pipeline into a single SaaS product. v1 delivers a working 2D prototype; v2 is the full enterprise platform ready for commercial deployment.

Contents

Version History

Version	Scope	Timeline	Key Additions
v1.0	2D Simulator MVP	2 months	Python, Pygame, OpenCV, PID control
v2.0	Enterprise SaaS Platform	12 months	3D engine, ML pipeline, cloud deployment, web UI, API, multi-tenant
v3.0	(Roadmap)	+6 months	Hardware-in-the-loop, real sensor SDK, marketplace

VERSION 1 — Intern Prototype (2 Months)

Project Overview

AutoStack v1 is a 2D top-down autonomous car simulator in Python. It implements the full AV pipeline — sense, perceive, plan, control — on any laptop using only pip-installable libraries. Goal: working demo, strong learning, foundation for v2.

Tech Stack

Tool	Purpose
Python 3.10+	Core language
Pygame	2D world, rendering
OpenCV	Lane detection
NumPy	Math/sensor
Matplotlib	Dashboard plots

Architecture

```
World (Pygame) --> Sensor --> Perception --> Planner (FSM) -->
   Controller (PID) --> Dashboard
```

`world.py` Pygame window, road tiles, obstacles, car sprite, game loop.
`sensor.py` Simulated camera crop + proximity circle cast.
`perception.py` OpenCV Canny + HoughLinesP lane detection; obstacle distance.
`planner.py` FSM: FOLLOW_LANE / AVOID_OBSTACLE / STOPPED.
`controller.py` PID converts targets to per-frame movement deltas.
`dashboard.py` Live overlay: speed, steering, FSM state, proximity bar.

8-Week Timeline

Week	Milestone
1	Repo setup, Pygame window, road + car, keyboard control
2	Moving obstacles, collision detection, track wraparound
3	sensor.py — camera crop, proximity ring
4	perception.py — lane detection, draw detected lines
5	planner.py — FSM, autonomous lane following
6	Obstacle avoidance, FSM transitions
7	Dashboard overlay, all sensor data live
8	Polish, demo video, README, 5-min presentation

Deliverables & Resume Bullets (v1)

- GitHub repo, working simulator, screen-recorded demo, 5-min presentation
- *Built a 2D AV simulator in Python with OpenCV lane detection and PID-controlled obstacle avoidance*
- *Implemented perception-planning-control pipeline with live sensor dashboard using Pygame + Matplotlib*

VERSION 2 — Enterprise SaaS Platform (12 Months)

Market Opportunity

Problem Statement

AV and ADAS development teams spend 60–70% of their validation budget on physical test drives. Simulation reduces cost but existing tools (CARLA, NVIDIA DRIVE Sim) require dedicated GPU servers, proprietary licenses, or deep robotics expertise. Small teams and startups are priced out. Cloud-native, ML-integrated simulation accessible via a web browser does not exist as a turnkey product.

Target Customers

Segment	Profile	Pain AutoStack Solves
ADAS Startups (10–100 eng)	Building camera/radar perception stacks	No GPU lab, need fast scenario iteration
Automotive OEM R&D Labs	Testing Level 2–4 features	Reduce physical test miles by 40%
Robotics Companies	Warehouse/delivery robots	Adapt AV simulation to robot kinematics
University Research Labs	PhD AV/ML research	Free tier, reproducible experiments
Defence/Aerospace	Autonomous UAV path planning	Secure on-prem deployment option

Competitive Landscape

Product	3D Sim	ML CI/CD	Cloud-Native	Price
CARLA (Open Source)	Yes	No	No	Free but GPU required
NVIDIA DRIVE Sim	Yes	Partial	No	$100k+ enterprise
Cognata	Yes	No	Yes	$50k+/yr
Waymo Simulation	Yes	Yes	Internal only	Not for sale
AutoStack v2	**Yes**	**Yes**	**Yes**	**SaaS, affordable**

Unique Differentiators

- **No GPU required** — 3D sim runs in cloud; client is a browser
- **ML Validation CI/CD** — perception model accuracy gated on every PR
- **Scenario-as-Code** — scenarios defined in JSON, version-controlled
- **Open Core** — simulation engine open-source; platform services commercial
- **Multi-tenant SaaS** — team isolation, RBAC, usage-based billing

Product Tiers & Pricing

Feature	Free	Pro ($299/mo)	Enterprise (Custom)
Concurrent sim runs	1	10	Unlimited
Scenario library	5 built-in	50+ community	Custom + private
ML validation pipeline	No	Yes	Yes + custom models
Storage (S3)	1 GB	50 GB	Unlimited
Data retention	7 days	90 days	Unlimited
SSO / SAML	No	No	Yes
On-premise deployment	No	No	Yes
SLA	None	99.5%	99.9%
Support	Community	Email (48h)	Dedicated CSM

Enterprise Architecture

System Overview

```
Browser (React + Three.js)
  |  REST / WebSocket
  v
[API Gateway (Kong)]
  |  JWT auth, rate limit, tenant routing
  v
[Core Services (FastAPI, EKS)]
  |-- Scenario Service   -- CRUD scenarios, assets
  |-- Simulation Service -- spin up/down sim workers
  |-- ML Service         -- model registry, inference, MLflow
  |-- Telemetry Service  -- ingest + stream real-time data
  +-- Auth Service       -- JWT, RBAC, tenant management
  |
  v
[Job Queue (Celery + Redis)]
  |  async sim jobs, ML jobs
  v
[Simulation Workers (EKS pods)]
  |-- 3D Engine (Panda3D / Ursina)
  |-- Sensor Fusion (NumPy + Kalman)
  |-- ML Perception (YOLOv8, PyTorch)
  +-- Planner + Controller
  |
[PostgreSQL]     [S3 (scenarios, models, reports)]
[Redis Streams]  [CloudWatch + Prometheus + Grafana]
```

Component Descriptions

3D Simulation Engine

Panda3D rendering with road network loaded from GeoJSON-like scenario files. Physics: simple rigid body (PyBullet optional for advanced dynamics). Supports weather over-

lays (rain, fog as perception difficulty modifiers). Runs headless on EKS pods; streams rendered frames as JPEG via Redis to frontend WebSocket. Target: 30 FPS at 1280x720 per simulation instance.

Sensor Fusion Module

Simulates: forward camera (rendered frame crop), LIDAR (NumPy point cloud generation around vehicle), radar (range + velocity returns). Kalman Filter fuses camera + LIDAR detections into unified object state (position, velocity, class). Configurable noise model per sensor (Gaussian noise, dropout).

ML Perception Engine

YOLOv8-nano for real-time object detection on camera frames (runs on CPU at 15 FPS). Traffic sign classifier (ResNet-18 fine-tuned, ¡5MB). Inference results: bounding boxes, class labels, confidence, estimated distance. MLflow experiment tracking: model version, scenario set, mAP per class.

Advanced Planner

Behavior Tree (py_trees) replaces FSM. Nodes: sequence, selector, condition checks. Route planner: A* on road graph (nodes = intersections, edges = road segments with speed limits). Obstacle avoidance: RRT in local frame; re-plans every 500ms if obstacle detected. Decision outputs: target waypoint, speed profile, indicator signal.

Scenario Service

CRUD API for scenarios. Scenario defined in JSON: vehicle start/end, road network, obstacle placement, weather, sensor config, success/failure criteria. Versioned via S3 with hash-based deduplication. Community scenario marketplace: publish, fork, rate scenarios.

Telemetry Service

Simulation workers publish telemetry events to Redis Streams at 10Hz. Telemetry: position, speed, steering, sensor readings, planner state, perception results, collisions. Consumers: frontend (live WebSocket feed), PostgreSQL (persistent storage), anomaly detector. Supports replay: store telemetry, re-render at any speed.

API Reference

Authentication

All API requests require `Authorization: Bearer <JWT>` header. JWT contains: `tenant_id`, `user_id`, `role` (admin/engineer/viewer), `exp`.

Scenarios API

```
POST   /api/v1/scenarios              Create scenario
GET    /api/v1/scenarios              List scenarios (tenant-scoped)
GET    /api/v1/scenarios/{id}         Get scenario detail
PUT    /api/v1/scenarios/{id}         Update scenario
DELETE /api/v1/scenarios/{id}         Delete scenario
POST   /api/v1/scenarios/{id}/fork    Fork community scenario
```

Scenario Schema (POST body):

```
{
  "name": "highway_merge_rain",
  "description": "3-lane highway merge in heavy rain",
  "version": "1.0",
  "map": {
    "type": "highway",
    "lanes": 3,
    "length_m": 500,
    "speed_limit_kmh": 120
  },
  "weather": { "type": "rain", "intensity": 0.8 },
  "ego_vehicle": {
    "start": {"x": 0, "y": 1, "heading_deg": 0},
    "goal":  {"x": 450, "y": 0}
  },
  "obstacles": [
    {"type": "car", "x": 100, "y": 1, "speed_kmh": 80, "behavior": "
       lane_change"},
    {"type": "truck", "x": 200, "y": 2, "speed_kmh": 60, "behavior":
       "constant"}
  ],
  "sensors": {
    "camera": {"fov_deg": 90, "noise": 0.05},
    "lidar": {"range_m": 50, "points_per_scan": 360, "noise": 0.02}
  },
  "success_criteria": {
    "reach_goal": true,
    "max_collisions": 0,
    "max_time_sec": 60
  }
}
```

Simulation Runs API

```
POST /api/v1/runs                     Start simulation run
GET  /api/v1/runs                     List runs (filter by scenario,
   status, date)
GET  /api/v1/runs/{id}                Get run detail + metrics
GET  /api/v1/runs/{id}/telemetry      Stream telemetry (WebSocket)
GET  /api/v1/runs/{id}/replay         Get replay data
POST /api/v1/runs/{id}/cancel         Cancel in-progress run
```

Run Response Schema:

```
{
  "id": "run_abc123",
  "scenario_id": "scen_xyz",
```

```
  "status": "running|completed|failed|cancelled",
  "started_at": "2026-05-22T14:00:00Z",
  "completed_at": "2026-05-22T14:01:03Z",
  "duration_sec": 63,
  "metrics": {
    "outcome": "success|collision|timeout",
    "distance_m": 447.2,
    "avg_speed_kmh": 98.4,
    "min_obstacle_distance_m": 8.2,
    "perception_accuracy": { "car": 0.91, "truck": 0.87 },
    "collisions": 0,
    "lane_departures": 1
  },
  "model_version": "yolov8n-v2.1"
}
```

ML Models API

```
POST /api/v1/models                    Register model version
GET  /api/v1/models                    List model versions
GET  /api/v1/models/{id}/metrics       Get accuracy metrics per
   scenario
POST /api/v1/models/{id}/validate      Run validation suite (async)
GET  /api/v1/models/{id}/compare       Compare two model versions
```

Database Schema

Core Tables

```
-- Tenant isolation
CREATE TABLE tenants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  plan        TEXT NOT NULL CHECK (plan IN ('free','pro','enterprise
     ')),
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Users
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  email       TEXT NOT NULL UNIQUE,
  role        TEXT NOT NULL CHECK (role IN ('admin','engineer','
     viewer')),
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Scenarios
CREATE TABLE scenarios (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  name        TEXT NOT NULL,
  description TEXT,
```

```
  version     TEXT NOT NULL DEFAULT '1.0',
  config      JSONB NOT NULL,       -- full scenario JSON
  is_public   BOOLEAN DEFAULT false,
  created_by  UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- Simulation Runs
CREATE TABLE runs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  scenario_id   UUID NOT NULL REFERENCES scenarios(id),
  model_id      UUID REFERENCES ml_models(id),
  status        TEXT NOT NULL DEFAULT 'queued',
  started_at    TIMESTAMPTZ,
  completed_at  TIMESTAMPTZ,
  outcome       TEXT,               -- success|collision|timeout
  metrics       JSONB,              -- full metrics JSON
  worker_node   TEXT,               -- EKS pod that ran it
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- Telemetry (partitioned by run_id for performance)
CREATE TABLE telemetry (
  id        BIGSERIAL,
  run_id    UUID NOT NULL REFERENCES runs(id),
  ts        TIMESTAMPTZ NOT NULL,
  frame_no  INT NOT NULL,
  data      JSONB NOT NULL     -- position, speed, sensor readings,
     detections
) PARTITION BY HASH (run_id);

-- ML Models
CREATE TABLE ml_models (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  name          TEXT NOT NULL,
  version       TEXT NOT NULL,
  type          TEXT NOT NULL,   -- perception|planning|control
  s3_path       TEXT NOT NULL,
  mlflow_run_id TEXT,
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- Validation Results (ML CI/CD)
CREATE TABLE validation_results (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  model_id    UUID NOT NULL REFERENCES ml_models(id),
  run_ids     UUID[] NOT NULL,   -- scenario runs used for validation
  map_score   FLOAT NOT NULL,    -- mean average precision
  baseline_id UUID REFERENCES ml_models(id),
  passed      BOOLEAN NOT NULL,  -- true if map >= threshold
  report_url  TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Indexes
```

```
CREATE INDEX idx_runs_tenant ON runs(tenant_id, created_at DESC);
CREATE INDEX idx_scenarios_tenant ON scenarios(tenant_id, is_public);
CREATE INDEX idx_telemetry_run ON telemetry(run_id, ts);
```

Security Architecture

Threat Model

Threat	Impact	Control
Cross-tenant data leak	Competitor sees your scenarios/models	Row-level tenant_id filter on all queries; enforced at ORM layer
ML model IP theft	Proprietary perception model stolen	S3 pre-signed URLs (15min expiry); no direct S3 access
Simulation worker escape	Worker accesses other tenants' data	Workers run in isolated EKS namespaces; no DB access directly
API token compromise	Full account access	JWT 1hr expiry; refresh token rotation; IP-based anomaly alerts
Prompt injection in scenario config	Malicious scenario breaks worker	Scenario JSON schema validated via Pydantic; no eval/exec

Security Controls

- **Encryption**: TLS 1.3 in transit; AES-256 at rest (RDS, S3)
- **Secrets**: All credentials in AWS Secrets Manager; zero in environment variables
- **IAM**: EKS service accounts with least-privilege IRSA roles per service
- **Network**: VPC with private subnets; no direct internet access from workers
- **SAST**: Bandit + Semgrep in CI on every PR
- **Container**: Trivy scan on every Docker image before deploy
- **Audit Log**: All API calls logged to CloudWatch with tenant_id + user_id

Cloud Infrastructure & Cost Estimate

AWS Architecture

```
Region: ap-south-1 (Mumbai) -- primary
        us-east-1           -- DR (enterprise tier)

VPC:
  Public subnet:  ALB, NAT Gateway
  Private subnet: EKS worker nodes, RDS, ElastiCache

EKS Node Groups:
  system-ng:   2x t3.medium (API, auth, gateway)  always-on
  worker-ng:   0-20x c5.xlarge (sim workers)      auto-scale
```

```
  ml-ng:       0-10x c5.2xlarge (ML inference)    auto-scale

RDS: PostgreSQL 15, db.r6g.large, Multi-AZ, 500GB gp3
ElastiCache: Redis 7, cache.r6g.large, 1 replica
S3: Scenario assets, model weights, telemetry archives
CloudFront: Static React assets, scenario asset delivery
```

Monthly Cost Estimate (Pro Tier, 50 active users)

Resource	Cost/Month
EKS system nodes (2x t3.medium, reserved)	$60
EKS sim workers (avg 3x c5.xlarge, on-demand)	$450
RDS PostgreSQL (db.r6g.large, Multi-AZ)	$280
ElastiCache Redis (cache.r6g.large)	$120
S3 (500GB + transfer)	$25
ALB + CloudFront + data transfer	$40
CloudWatch + monitoring	$30
Total	**$1,005/mo**
Revenue at $299/user x 50	**$14,950/mo**
Gross Margin	**~93%**

ML Validation CI/CD Pipeline

```
# .github/workflows/ml-validation.yml
Trigger: PR to perception/ directory

Steps:
1. Build Docker image with new model weights
2. Push to ECR
3. POST /api/v1/models (register new version)
4. POST /api/v1/models/{id}/validate
   -- runs 20 regression scenarios
   -- computes mAP vs baseline
5. Poll until complete (max 15 min)
6. If mAP < baseline - 2%: fail PR check
7. Post validation report as PR comment
   -- per-class accuracy table
   -- worst-performing scenarios
   -- link to MLflow experiment
```

Validation Scenario Set (20 scenarios): highway_clear, highway_rain, highway_fog, highway_night, city_intersection, city_pedestrian, city_cyclist, city_bus, parking_lot, roundabout, construction_zone, emergency_vehicle, lane_change, emergency_brake, low_light_pedestrian, adverse_weather_lidar, sensor_dropout_50pct, multi_obstacle_dense, opposite_traffic, tunnel_exit.

12-Month Build Roadmap

Month	Phase	Deliverable
1–2	3D Engine + Sensor Fusion	Panda3D world, multi-agent, LIDAR/camera sim, Kalman filter
3–5	ML Perception + MLflow	YOLOv8 inference, sign classifier, experiment tracking
4–6	Advanced Planner	Behavior Tree, A* routing, RRT avoidance
5–7	Backend API + Scenario DB	FastAPI services, PostgreSQL schema, S3 scenario library
6–8	Web Dashboard	React + Three.js, live 3D, scenario browser, telemetry charts
7–9	ML CI/CD Pipeline	GitHub Actions validation, accuracy gating, PR reports
9–11	Cloud Deployment	EKS, RDS, Terraform, Helm, CloudWatch, autoscaling
11–12	Hardening + Launch	Pentest, load test, billing integration, public launch

Team Structure & Ownership

Role	Who	Owns
Architect/Lead	Somasekhar Eruvuri	Platform design, cloud infra, all contracts
Intern A	CS Intern	3D engine, sensor fusion, planner
Intern B	CS Intern	Backend API, scenario DB, ML CI/CD
Intern C	CS Intern	ML perception, web dashboard, MLflow

KPIs & Success Metrics

Metric	Target
Sim throughput	20 concurrent runs, no degradation
ML perception mAP (validation set)	¿85%
API p99 latency	¡200ms
CI validation pipeline time	¡15 min
Infrastructure provision time	¡15 min via Terraform
Platform uptime	¿99.5%
Free-to-Pro conversion	¿8% within 60 days

Risk Register

Risk	Impact	Mitigation
3D engine CPU on EKS	Cost spike if sim is slow	Profile early; headless render budget ¡4 vCPU/run

YOLOv8 latency ¿100ms	Real-time loop breaks	Use YOLOv8-nano; run async with bounded queue
AWS cost overrun	Platform unprofitable at low scale	Auto-shutdown idle workers; budget alarm at 80%
Intern knowledge gaps	ML/cloud modules delayed	Architect provides starter templates; weekly pairing sessions

v3 Roadmap (Post-Launch)

- **Hardware-in-the-loop** — physical ECU connects to simulation via CAN bus bridge
- **Real sensor SDK** — import real LIDAR/camera recordings as simulation inputs
- **Scenario Marketplace** — community-contributed scenarios with rating/download metrics
- **Federated Learning** — multiple teams improve shared perception model without sharing raw data
- **Regulatory Compliance Packs** — pre-built scenario sets for ISO 26262, UN Regulation 157

Resume Bullet Points (v2)

- Architected a cloud-native AV simulation SaaS on AWS EKS supporting 20 concurrent 3D simulation runs with YOLOv8 ML perception, scenario-as-code versioning, and a React + Three.js live telemetry dashboard
- Designed an ML validation CI/CD pipeline in GitHub Actions that runs 20 regression scenarios on every PR and gates merges on perception accuracy benchmarks tracked in MLflow, with automated PR reporting
- Built a multi-tenant platform with JWT + RBAC, row-level tenant isolation, S3 model/asset storage, and Terraform-provisioned EKS infrastructure with Prometheus/Grafana observability
- Led product design from prototype to commercial SaaS: defined API contracts, PostgreSQL schema, pricing tiers, threat model, and AWS cost structure achieving ¿90% gross margin at Pro tier